语义层级	IREE 原生阶段	CIM COMPILER
前端接入 与高层语义优化	**01 input** 输入转换与公共规范化	**线性算子规范化：** 统一 ONNX 导入产生的线性计算表达。 **前端语义收敛：** 输出可继续进入 IREE 优化的张量 IR。
	02 abi	
	03 preprocessing 用户与插件预处理	**图级重组：** 异构块对角融合后调用原生横向融合，保留 M<16 限制。
	04 global-optimization dispatch 前的全局张量优化	**原生张量优化：** 沿用上游折叠与清理。 **阶段复用：** 不增加 global-optimization hook。
计算分组 与执行调度	**05 dispatch-creation** 融合计算并形成 dispatch 边界	**原生任务形成：** 按数据依赖建立 dispatch 边界，无自定义 hook。
	06 flow outline dispatch 为 target executable	
	07 stream 资源、时间线与命令调度	**原生调度复用：** 保持 Flow 的 executable 划分和 Stream 的资源时序。
目标配置 与代码生成	**08 executable-sources** 设备分配与 executable 接口	**目标后端注册：** 声明 CIM 后端、可执行格式和支持类型。 **硬件配置：** 将 Array 与 SRAM 参数绑定到目标编译。
	09 executable-configurations 选择目标 lowering 策略	**Lowering 配置：** 选择目标翻译策略与 workgroup 划分。 **分块选择：** 确定满足 Array 约束的 M/N/K 整数切分。
	10 executable-targets 翻译 executable 为目标 IR	**IR lowering：** 完成 workgroup 分块、融合计算分解和二维合法化。 **缓冲区化：** 沿 IREE 路径物化 dispatch 张量和静态常量。 **IR 转换：** 将 Linalg 降为合法的逻辑 CIM 程序。
部署物理化 与可执行文件封装	**11 hal** host HAL 转换、链接与序列化	**逻辑权重优化：** 执行横向拼接与可选权重复制。 **Array 部署：** 统一完成权重去重、切分、放置和一次性初始化。 **SRAM 管理：** 物化数据搬运与临时缓冲，并按生命周期复用空间。 **指令物理化：** 展开 bit-serial 计算并完成 affine 地址 lowering。 **可执行封装：** 从同一物理程序生成可执行载荷与 ISA。
主机程序 Lowering	**12 vm** 主机程序 lowering 为 VM module	**运行时衔接：** CIM executable 作为二进制进入原生 VM 主机程序。